NES Scheduling Engine

Finalist Discovery Sprint Brief

New England SteamWorks

Sprint type	Paid discovery sprint only — not the full implementation contract
Time cap	15 hours
Purpose	Evaluate understanding, structure, Version 1 judgment, and maintainability
Finalists	All finalists receive the same sprint scope for comparison

This sprint is for discovery and architecture only. The goal is to evaluate how you translate a rules-heavy weekly dispatch-planning problem into a practical Python engine design without overbuilding it.

1) Project context

New England SteamWorks is a residential service business operating across a large geographic territory. This project is not a booking app, chatbot, or standard calendar workflow. It is a weekly dispatch-planning and route-grouping engine.

- which candidate jobs should be scheduled this week
- how they should be grouped geographically
- which technician / vehicle combinations can perform them
- which jobs should remain in reserve / standby
- which areas are sufficiently ready to justify a run

The system should be rules-driven, practical, maintainable, understandable by a non-technical owner, and easy to modify over time. Human review is part of the workflow, but the engine should not depend on heavy weekly manual repair.

2) System boundaries

- ServiceM8 = system of record
- Airtable = scheduling rules, structured inputs, review layers, and outputs
- Make / Zapier = automation and orchestration
- Python engine = the scope of this role

This sprint is specifically about the external Python scheduling engine, not the Airtable build or automation layer.

3) Core scheduling model

Weekly, route-first planning

The engine should analyze one full week at a time, not day-by-day. It should preprocess the candidate pool, subtract exclusions, generate real weekly route capacity, check special-route needs first, build route/day proposals, and output maps, backups, and area readiness for review.

Hybrid geography

Operations are route-first and geography-aware, but customer communication remains area-based. Area structure matters, but area boundaries are guidance, not hard walls. Neighboring areas should be mixed when geography truly supports a better route.

Seasonal fairness

Fairness and aging pressure matter, but not equally in all seasons. In March–September, clean geography should dominate more strongly. In October–February, fairness and aging should matter more aggressively while still respecting geographic sanity.

Human review, not auto-booking

The engine proposes a draft schedule. It does not write directly into the ServiceM8 calendar, does not promise dates early, and does not assign exact stop order or appointment times.

4) Important business rules and constraints

Candidate pool

Assume the engine receives a prefiltered candidate pool of jobs eligible for scheduling. Already scheduled, canceled, urgent/manual, already-in-workflow, or otherwise excluded jobs should be out of the pool before normal analysis begins.

Weekly exceptions

Capacity is affected by weekly exceptions such as technician unavailability, vehicle unavailability, pre-scheduled long jobs, boiler installs already consuming capacity, and similar known constraints. Weekly availability should come from weekly exceptions, not from technician master data alone.

Special route types

Handle special route types before normal route filling, including Radiator Runs, NH Overnight Runs, and Two-Man / helper-needed routes.

Vehicle rules

V-2 must be used for Radiator Runs. V-4 is a weekly capacity switch. V-5 / V-6 / V-7 are standard service vehicles. The engine should generate weekly capacity first, then fill it.

Backup / standby jobs

The engine must output both assigned jobs and standby jobs separately. Standby jobs are plausible substitute jobs held for downstream workflow if something drops out.

Duplicate-address handling

Detect duplicate or same-address job groups and flag them, but do not silently delete them. Multiple jobs at one address should not inflate stop density for route-shape purposes.

Area Readiness

Area Readiness remains a communication layer and must stay credible relative to actual run behavior. Allowed values are Good, Moderate, and Lean. The engine should also output a plain-text lookup block for downstream use.

Maps are required

Each proposed route/day must have a plotted map. Maps are a required review artifact, not an optional enhancement.

5) Sprint objective

Please use this sprint to produce a discovery package that addresses:

1. your understanding of the planning problem
2. your recommended Python engine structure
3. what belongs in code vs config vs Airtable-managed policy
4. what should be treated as hard constraints vs scored / weighted logic
5. how geography, fairness, area readiness, and route practicality should interact
6. what a realistic Version 1 should and should not do
7. biggest risks, ambiguities, and assumptions
8. how you would validate and test the engine given limited historical engine-style data

6) Deliverables

A. Discovery memo

- your understanding of the system
- recommended engine structure
- code vs config vs Airtable split
- hard constraints vs weighted logic
- Version 1 recommendation
- major risks / ambiguities
- validation / testing approach

B. Rule classification

- hard constraints
- eligibility filters
- weighted / preference logic
- owner-controlled policy inputs
- review flags / exception outputs

C. Small technical sketch

- pseudocode
- module outline
- schema sketch
- rule-evaluation structure
- weekly run flow

D. Short final recommendation summary

- your recommended Version 1
- biggest pitfall to avoid
- what should be clarified first before build

7) What not to do

- a full production build
- broad UI design
- workflow automation design outside the engine boundary
- exact stop sequencing or appointment-time logic
- speculative ML / AI features
- overbuilt optimization architecture unless clearly necessary for Version 1

A full prototype is not required.

8) Version 1 mindset

A strong response will show practical system framing, clear rule decomposition, restraint, maintainability, and the ability to support a non-technical owner. This should behave like a rules-driven operational tool, not a black box.

9) Question policy

Please work from the materials provided, make reasonable assumptions where needed, and state those assumptions clearly.

- send questions in one batch only
- maximum 5 questions
- prioritize only the highest-impact ambiguities
- if a point is unclear but does not materially change your overall approach, make your assumption explicit and proceed

10) Time cap

This sprint is capped at 15 hours. All finalists are receiving the same sprint scope for comparison.